

LAPORAN PRAKTIKUM
STRUKTUR DATA
“*ARRAYLIST*”
PADA BAHASA PEMROGRAMAN JAVA
disusun Oleh :
RAZIF AL FARISI
2511532028

Dosen Pengampu : Dr. Wahyudi S.T, M.T
Asisten Laboratorium : Sherly Sukmadira



DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
PADANG
2026

KATA PENGANTAR

uji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya, penulis dapat menyelesaikan laporan praktikum yang berjudul “*Stack pada Bahasa Pemrograman Java*” dengan baik dan tepat waktu.

Laporan ini disusun sebagai salah satu bentuk pertanggungjawaban kegiatan praktikum pada mata kuliah Struktur Data, khususnya mengenai penggunaan struktur data *Stack* dalam bahasa pemrograman Java. Praktikum ini bertujuan untuk memberikan pemahaman kepada mahasiswa mengenai konsep struktur data *LIFO (Last In First Out)* serta implementasinya dalam program.

Penulis menyadari bahwa penyusunan laporan ini tidak terlepas dari bimbingan, bantuan, dan dukungan dari berbagai pihak. Oleh karena itu, penulis mengucapkan terima kasih kepada dosen pengampu mata kuliah Struktur Data yang telah memberikan ilmu dan arahan selama proses pembelajaran.

Padang, 20 April 2026

Penulis

Daftar Isi

KATA PENGANTAR	i
BAB I.....	3
PENDAHULUAN	3
1.1 Latar Belakang.....	3
1.2 Tujuan	3
BAB II	4
PEMBAHASAN	4
2.1 Landasan Teori	4
2.2 Implementasi Program	4
2.2.1 Class Mahasiswa	4
2.2.2 Class Mahasiswa Driver	5
2.2.3 Class DaftarKata	6
2.2.4 Class DaftarKataDriver.....	7
2.3 Tugas Praktikum	8
2.3.1 Class Musik	8
2.3.2 Class Playlist (Driver Musik)	9
BAB III.....	12
PENUTUP.....	12
3.1 Kesimpulan.....	12
3.2 Saran	12
Referensi.....	13

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam pemrograman, pengelolaan data sangat penting untuk memastikan program berjalan dengan efisien. Salah satu struktur data yang sering digunakan adalah *Stack*.

Stack merupakan struktur data linear yang bekerja dengan prinsip *LIFO* (*Last In First Out*), yaitu elemen terakhir yang masuk akan menjadi elemen pertama yang keluar. Struktur ini banyak digunakan dalam berbagai aplikasi seperti evaluasi ekspresi matematika, *undo-redo*, dan pemanggilan fungsi (*call stack*).

Dengan memahami konsep *Stack*, programmer dapat mengelola data secara terstruktur dan efisien dalam berbagai permasalahan komputasi.

1.2 Tujuan

Tujuan dari penyusunan laporan praktikum ini adalah sebagai berikut:

1. Memahami konsep *Stack* dalam struktur data
2. Mengimplementasikan *Stack* menggunakan array
3. Memahami operasi dasar *Stack* seperti *push*, *pop*, dan *peek*
4. Mengimplementasikan *Stack* dalam kasus nyata seperti postfix dan pencarian nilai maksimum

BAB II

PEMBAHASAN

2.1 Landasan Teori

Stack adalah struktur data yang bekerja dengan prinsip *Last In First Out (LIFO)*.

Karakteristik *Stack* :

- Bersifat linear
- Akses hanya melalui satu sisi (*top*)
- Elemen terakhir masuk akan keluar pertama

Beberapa method yang umum digunakan pada *Stack* :

- *push()* : menambah elemen
- *pop()* : menghapus elemen teratas
- *peek()* : melihat elemen teratas
- *isEmpty()* : mengecek apakah kosong

2.2 Implementasi Program

Berikut adalah beberapa implemenentasi *ArrayList* :

2.2.1 Class StackArray

```
package pekan3_2511532028;

public class stackArray_2511532028 {
    static final int max = 1000;
    int top;
    int a[] = new int[max];

    boolean isEmpty_2511532028()
    {
        return (top < 0);
    }

    stackArray_2511532028()
    {
        top = -1;
    }

    boolean push_2511532028(int x)
    {
        if (top >= (max - 1)) {
            System.out.println("Stack Overflow");
            return false;
        }
        else {
            a[++top] = x;
            System.out.println(x+ " dimasukkan dalam stack");
            return true;
        }
    }
}
```

```

    }
    int pop_2511532028()
    {
        if (top < 0) {
            System.out.print("Stack Underflow");
            return 0;
        }
        else {
            int x = a[top--];
            return x;
        }
    }
    int peek_2511532028()
    {
        if (top < 0) {
            System.out.println("Stack Underflow");
            return 0;
        }
        else {
            int x = a[top];
            return x;
        }
    }
    void print_2511532028() {
        for (int i = top; i > -1; i--) {
            System.out.print(" " + a[i]);
        }
    }
}

```

Class ini merupakan implementasi *Stack* menggunakan array.

Variable :

- *max* : kapasitas maksimum stack
- *top* : penunjuk elemen teratas
- *a[]* : array penyimpan data

Method :

- *push()* : menambah data ke stack
- *pop()* : menghapus data teratas
- *peek()* : melihat data teratas
- *isEmpty()* : mengecek kosong atau tidak
- *print()* : menampilkan isi stack

2.2.2 Class StackArrayDriver

```

package pekan3_2511532028;

public class stackArrayDriver_2511532028 {

    public static void main(String[] args) {
        stackArray_2511532028 s = new stackArray_2511532028();
        s.push_2511532028(10);
    }
}

```

```

        s.push_2511532028(20);
        s.push_2511532028(30);
        System.out.print("Elemen pada Stack : ");
        s.print_2511532028();
        System.out.println("\n" + s.pop_2511532028() + " dikeluarkan dari
stack ");
        System.out.println("Elemen teratas adalah : " +
s.peek_2511532028());
        System.out.print("Elemen pada stack : ");
        s.print_2511532028();
    }
}

```

Program ini digunakan untuk menguji operasi *Stack*.

Alur :

1. Menambahkan data ke stack (*push*)
2. Menampilkan isi stack
3. Menghapus elemen (*pop*)
4. Melihat elemen teratas (*peek*)

2.2.3 Class StackPostfix

```

package pekan3_2511532028;

import java.util.Stack;
import java.util.Scanner;

public class StackPostfix_2511532028 {
    public static int postfixEvaluate_2511532028( String expression ) {
        Stack<Integer> s = new Stack<Integer>();
        Scanner input = new Scanner(expression);
        while (input.hasNext()) {
            if (input.hasNextInt()) {
                s.push(input.nextInt());
            } else {
                String operator = input.next();
                int operand2 = s.pop();
                int operand1 = s.pop();
                if (operator.equals("+")) {
                    s.push(operand1 + operand2);
                } else if (operator.equals("-")) {
                    s.push(operand1 - operand2);
                } else if (operator.equals("*")) {
                    s.push(operand1 * operand2);
                } else {
                    s.push(operand1 / operand2);
                }
            }
        }
        input.close();
        return s.pop();
    }
    public static void main(String[] args) {

```

```

        System.out.println("hasil postfix = " +
postfixEvaluate_2511532028("5 2 4 * + 7 -"));
    }
}

```

Program ini digunakan untuk menghitung ekspresi *postfix*.

Variabel :

- Jika angka → push ke stack
- Jika operator → ambil 2 angka dari stack
- Lakukan operasi
- Simpan kembali ke stack
- Hasil akhir diambil dari stack

2.2.4 Class NilaiMaksimum

```

package pekan3_2511532028;

import java.util.Stack;

public class NilaiMaksimum_2511532028 {
    public static int max_2511532028(Stack<Integer> s) {
        Stack<Integer> backup = new Stack<Integer>();
        int maxValue = s.pop();
        backup.push(maxValue);
        while (!s.isEmpty()) {
            int next = s.pop();
            backup.push(next);
            maxValue = Math.max(maxValue, next);
        }
        while (!backup.isEmpty()) {
            s.push(backup.pop());
        }
        return maxValue;
    }

    public static void main(String[] args) {
        Stack<Integer> s = new Stack<Integer>();
        s.push(70);
        s.push(12);
        s.push(20);
        System.out.println("Isi stack      : " + s);
        System.out.println("Stack Teratas   : " + s.peek());
        System.out.println("Nilai Maksimum : " + max_2511532028(s));
    }
}

```

Program ini digunakan untuk mencari nilai maksimum dalam stack tanpa menghilangkan data.

Alur :

1. Ambil elemen pertama sebagai nilai awal

2. Bandingkan dengan elemen lain
3. Simpan di stack sementara
4. Kembalikan data ke stack awal
5. Tampilkan nilai maksimum

2.3 Tugas Praktikum

2.3.1 Class Website_2511532028

```
package pekan3_2511532028;

public class Website_2511532028 {
    String Judul_2028;
    String URL_2028;
    int top = 0;
    //static final int max = 20;

    public Website_2511532028(String judul, String url) {
        this.Judul_2028 = judul; this.URL_2028 = url;
    }

    // mutator
    public void setJudul(String judul) {this.Judul_2028 = judul;}
    public void setUrl(String url) { this.URL_2028 = url;}

    // selektor
    public String getJudul() {return Judul_2028;}
    public String getUrl() {return URL_2028;}

    // menghindari terjadinya tampilan hashcode
    public String toString() {
        return " Judul      : " + Judul_2028 + "\n URL: " + URL_2028;
    }
}
```

Class ini digunakan untuk merepresentasikan sebuah *website* sebagai objek yang akan disimpan dalam *Stack*.

Variable :

- judul_2028 : menyimpan judul website
- URL_2028 : menyimpan alamat website
- top : (tidak digunakan dalam logika utama, karena stack menggunakan class bawaan)

Method :

- Konstruktor : Digunakan untuk menginisialisasi objek website saat pertama kali dibuat
- setJudul() : mengubah judul website
- setUrl() : mengubah URL

- `getJudul()` : mengambil judul
- `getUrl()` : mengambil URL
- `toString()` : menampilkan informasi lagu

2.3.2 Class Browser (Driver Website)

```
package pekan3_2511532028;

import java.util.Stack;
import java.util.Scanner;

public class Browser_2511532028 {

    // pilihan menu pada browser
    public static void tampilkanMenuBrowser_2511532028() {
        System.out.println("\n====Menu Browser====");
        System.out.println("1. Kunjungi Website");
        System.out.println("2. Balik ke Website Sebelumnya");
        System.out.println("3. Lihat Halaman Aktif");
        System.out.println("4. Keluar");
    }

    // Untuk mengunjungi website dengan judul dan URL
    public static void kunjungiWebsite_2028(Stack<Website_2511532028> Web,
Scanner sc) {
        String judul;
        String url;

        System.out.print("\nMasukkan Judul Web : ");
        judul = sc.nextLine();

        System.out.print("Masukkan URL          : ");
        url = "https://" + sc.nextLine();

        Web.push(new Website_2511532028(judul , url));
        System.out.println("Website Dibuka");
    }

    // perintah kembali ke website sebelumnya
    public static void tombolbackWebsite_2028(Stack<Website_2511532028> Web) {
        if (!Web.isEmpty()) {
            Web.pop();
            System.out.println("---(Anda balik ke Website sebelumnya)---");
        } else {
            System.out.println("---(Anda belum mengunjungi Website apapun)---");
        }
    }

    // menampilkan website yang saat ini dibuka
    public static void tampilWebsite_2028(Stack<Website_2511532028> Web) {
        if (!Web.isEmpty()) {
            System.out.println("\n====Website saat ini====");
            System.out.println(Web.peek().toString());
        } else {
            System.out.println("---(Anda belum mengunjungi Website apapun)---");
        }
    }
}
```

```

    }
}

public static void main(String[] args) {
    Scanner i = new Scanner(System.in);
    Stack<Website_2511532028> Website = new Stack<Website_2511532028>();
    int choice;

    do {
        tampilkanMenuBrowser_2511532028();
        System.out.print("Pilih menu : ");
        choice = i.nextInt();
        i.nextLine();

        // Untuk memilih menu
        switch (choice) {
            case 1:
                kunjungiWebsite_2028(Website, i);
                break;
            case 2:
                tombolbackWebsite_2028(Website);
                break;
            case 3:
                tampilWebsite_2028(Website);
                break;
            case 4:
                System.out.println("---(Keluar dari program browser)---");
                break;
        }
    } while (choice != 5);
    i.close();
}
}

```

Program ini merupakan simulasi sederhana *browser* yang memanfaatkan struktur data *Stack*.

Variable :

- *Stack<Website_2511532028> Website* : digunakan untuk menyimpan riwayat website
- *Scanner i* : untuk menerima input dari pengguna
- *Choice* : menyimpan pilihan menu

Method :

1. Menu Program

Program menggunakan *looping do-while* untuk menampilkan menu :

- Kunjungi Website

- Kembali ke halaman sebelumnya
- Lihat halaman sekarang
- Keluar

2. Kunjungi Website

Digunakan untuk membuka website baru. Methodnya yaitu “`kunjungiWebsite()`” :

- Input judul dan *URL*
- *URL* otomatis ditambahkan “https://”
- Data dimasukkan ke stack menggunakan *push()*

3. Kembali ke halaman sebelumnya

Digunakan untuk kembali ke halaman sebelumnya. Merepresentasikan tombol back pada browser. Methodnya “`tombolbackWebsite()`” :

- Menghapus elemen teratas dengan *pop()*
- Jika kosong → tampil pesan

4. Lihat halaman sekarang

Menampilkan website yang sedang aktif:

- Menggunakan *peek()*
- Tidak menghapus data, hanya melihat elemen teratas

5. Sistem Kerja

1. Program dimulai dan menampilkan menu
2. User memilih menu
3. Program menjalankan sesuai pilihan :
 - Menu 1 → *push* website
 - Menu 2 → *pop* website
 - Menu 3 → *peek* website
4. Data website disimpan dalam Stack
5. Program berjalan terus dengan loop do-while
6. Program berhenti saat User memilih “keluar”

6. Konsep Stack pada Program

Stack itu prinsipnya *Last in First Out* (LIFO).

- Website terakhir dibuka itu berada di paling atas
- Saat tombol back di tekan, Website yang berada paling atas di hapus
- Website sebelumnya akan muncul,
- misalnya SSO Unand – Spotify – Steam, Steam adalah elemen paling atas. Saat tombol back ditekan, steam akan hilang/dihapus, otomatis website sebelumnya terbuka Kembali yaitu Spotify. Jadi, historynya SSO Unand – Spotify.

BAB III

PENUTUP

3.1 Kesimpulan

Stack merupakan struktur data yang menggunakan prinsip *LIFO*. Dalam Java, *Stack* dapat diimplementasikan menggunakan array maupun class bawaan.

Operasi dasar seperti *push*, *pop*, dan *peek* sangat penting dalam manipulasi data. Selain itu, *Stack* juga dapat digunakan dalam berbagai kasus seperti evaluasi postfix dan pencarian nilai maksimum

3.2 Saran

Dalam mempelajari *Stack*, disarankan untuk memahami konsep dasar terlebih dahulu sebelum mengimplementasikan program. Latihan yang beragam akan membantu meningkatkan pemahaman terhadap struktur data ini.

Referensi

Data Structures and Algorithms in Java

Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2014). *Data Structures and Algorithms in Java* (6th Edition). Wiley.

Oracle. *Java Collections Framework*.